

19.pdf



University of Engineering & Management

Document Details

Submission ID

trn:oid::3618:103124101

Submission Date

Jul 1, 2025, 2:33 PM GMT+5:30

Download Date

Jul 5, 2025, 12:44 AM GMT+5:30

File Name

19.pdf

File Size

796.6 KB

15 Pages**4,169 Words****23,271 Characters**

13% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **40 Not Cited or Quoted 11%**
Matches with neither in-text citation nor quotation marks
-  **1 Missing Quotations 0%**
Matches that are still very similar to source material
-  **5 Missing Citation 2%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 11%  Internet sources
- 8%  Publications
- 7%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 40** Not Cited or Quoted 11%
Matches with neither in-text citation nor quotation marks
- 1** Missing Quotations 0%
Matches that are still very similar to source material
- 5** Missing Citation 2%
Matches that have quotation marks, but no in-text citation
- 0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 11% Internet sources
- 8% Publications
- 7% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	www.irnetexplore.ac.in	2%
2	Publication	Ribeiro, Rui André Santos. "Concurrency Hot Spot Optimisation in Transactional ...	<1%
3	Internet	www.ijisae.org	<1%
4	Internet	inesc-id.pt	<1%
5	Internet	technodocbox.com	<1%
6	Publication	Maria Couceiro, Paolo Romano, Luis Rodrigues. "HPCS 2011 tutorials: Tutorial I: Di...	<1%
7	Internet	www.dpss.inesc-id.pt	<1%
8	Internet	www.slideserve.com	<1%
9	Submitted works	University of Edinburgh on 2011-08-18	<1%
10	Internet	www.gsd.inesc-id.pt	<1%

11	Internet	www.inesc-id.pt	<1%
12	Internet	www.sweetstudy.com	<1%
13	Internet	hdl.handle.net	<1%
14	Internet	arxiv.org	<1%
15	Internet	open.uns.ac.rs	<1%
16	Submitted works	Open University on 2025-04-10	<1%
17	Internet	bot.heartsupport.com	<1%
18	Publication	Gokarna Sharma, Brett Estrade, Costas Busch. "Chapter 7 Window-Based Greedy ...	<1%
19	Publication	Paolo Romano, Luis Rodrigues, Nuno Carvalho, João Cachopo. "Cloud-TM", ACM SI...	<1%
20	Publication	William N. Scherer. "Software transactional memory for dynamic-sized data struc...	<1%
21	Internet	www.freepatentsonline.com	<1%
22	Submitted works	University of Sheffield on 2016-02-12	<1%
23	Internet	insights2techinfo.com	<1%
24	Internet	www.coursehero.com	<1%

25	Internet	dblp.uni-trier.de	<1%
26	Internet	kipdf.com	<1%
27	Internet	www.dis.uniroma1.it	<1%
28	Publication	Koduru, Sai Charan. "Size Oblivious Programming of Clusters for Irregular Paralle...	<1%
29	Submitted works	University of Malaya on 2012-07-30	<1%
30	Internet	dblp.dagstuhl.de	<1%
31	Internet	ipfs.io	<1%
32	Internet	vts.uni-ulm.de	<1%
33	Publication	Miroslav Popovic, Branislav Kordic, Ilija Basicevic. "Transaction scheduling for Sof...	<1%

Comparison of GDP and Unemployment Calculation using Locks and Software Transactional Memory

Ribhu Ghosh¹, Shyama Mondal², Ryan Saptarshi Ray³

^{1, 2, 3}Government College of Engineering and Ceramic Technology
Kolkata, India

Abstract: Parallel processing is very important and commonly utilized these days. In parallel computing, locks are now used to solve synchronization problems. But there are several issues with locks. Software transactional memory, or STM, is an alternative approach to synchronization problems that does not have the drawbacks of locks. Since GDP growth gives a clear picture of an economy's performance across time, it is essential for economic study. Another important statistic for examining economic statistics is the unemployment rate. This paper examines the use of Software Transactional Memory (STM) to calculate GDP and Unemployment Rates. This "India GDP 1960-2022" dataset was taken as input. STM improves the accuracy and efficacy of GDP and Unemployment calculations. STM provides a way to manage shared memory in systems with multiple threads. This paper has discussed the challenges STM addresses and the potential benefits it offers in the context of economic calculations. Since STM performs similarly to locks and is unaffected by their flaws, it is better than locks. Locks adopt a pessimistic approach. STM is better, though, because it takes an optimistic approach. Additionally, by simply replacing lock calls with STM calls, codes with locks can be converted to codes with STM. Therefore, STM's ability to be used with old code is one of its main advantages. GDP and Unemployment rates for a range of years have been calculated simultaneously from "India GDP 1960-2022" dataset accessed from Kaggle. Simultaneous calculation improves effectiveness resulting in faster decision making.

Keywords: Parallel Processing, Synchronization Problem, Locks, Software Transactional Memory, Gross Domestic Product (GDP) Growth Calculation, Unemployment Rate Calculation

1 Introduction

The world is shifting from single processor architectures to multicore processors. Parallel apps are being created and CPU processing capacity is being increased to provide end users with additional processing power so they can do difficult and time-consuming tasks swiftly and easily. For multicore processors to be fully utilized, realistic parallel programming is needed.

Locks are currently employed to address the issue of synchronization. But among the issues with locks are convoying, priority inversion, deadlocks, and inefficient fault tolerance. Locks also adopt a pessimistic approach. Only one thread can be active at a time as a result.

2

Transactions have been a tried-and-tested method for handling concurrent computations in databases for a long time. Transactions allow for concurrent programming and eliminate locking-related problems. Transactional Memory (TM), a novel occurrence control prototype for multicore computers, is included into the parallel programming architecture through the integration of transactions. In contrast to conventional lock-based programming methods, TM aims to make parallel application development easier. One kind of TM that is not impacted by locks is software transactional memory (STM). Additionally, it utilizes optimistic execution, which performs better than locks by allowing multiple threads to run concurrently throughout the critical section.

Since GDP growth gives a clear picture of an economy's performance across time, it is essential for economic study. Here are some main justifications for its significance:

Economic Health Indicator: GDP growth indicates whether an economy is growing or shrinking. While a declining GDP may be a symptom of a recession or other economic difficulties, an increasing GDP usually indicates a robust, expanding economy.

Standard of Living Measurement: Gains in GDP might be seen as a sign of rising living standards. GDP growth frequently indicates increased production of products and services, which can result in better job prospects, higher incomes, and easier access to goods and services for individuals.

Policy Decisions: To influence monetary and fiscal policy, governments and central banks look to GDP growth. For example, authorities may **lower interest rates, increase government spending, or cut taxes** to boost the economy if GDP growth is slow. On the other hand, they may lower spending or raise interest rates to stop inflation if GDP is expanding too quickly.

Investment Decisions: When deciding where to deploy resources, investors consider GDP growth. Strong GDP growth could be a sign of a developing economy and an atmosphere that is conducive to economic success, which could raise stock prices and yield larger returns. Negative or sluggish growth may indicate stagnation or danger.

Labor Market Insights: Generally speaking, higher GDP results in more jobs being created, which lowers unemployment rates. The strength of the labor market and whether there is enough demand for workers to match economic output can be determined by analysts.

Inflation and Price Stability: Controlled inflation is frequently linked to steady GDP growth at a healthy rate. While standstill or negative growth may result in deflation,

24

3

too rapid growth may cause inflationary pressures, both of which are detrimental to the economy.

For a number of reasons, the unemployment rate is also a crucial indicator when examining economic data.

Economic Health Indicator: The unemployment rate is a useful tool for evaluating an economy's general health. Low unemployment points to a robust, working economy where jobs are being generated and the majority of job seekers can find employment, whereas high unemployment frequently denotes economic distress or labor market inefficiencies.

Labor Market Conditions: The economy's labor supply and demand are reflected in the unemployment rate. An increase in the unemployment rate may imply a slowdown in hiring, which could be an indication of structural problems in the labor market, a decline in consumer demand, or poorer business circumstances. On the other hand, a declining unemployment rate usually indicates robust labor demand and heightened economic activity.

Social and Political Impacts: Increased poverty, inequality, and social instability are some of the social repercussions of high unemployment. Because people may call for government action to address employment creation, it can also have an impact on political stability. Policymakers can better assess the social and political effects of economic performance by having a solid understanding of the unemployment rate.

Consumer Confidence and Spending: Unemployed people typically have less money available to them, which might diminish their spending and confidence. Changes in unemployment have a direct effect on economic growth since consumer spending accounts for a sizable amount of GDP. On the other hand, a low unemployment rate typically boosts consumer confidence and spending, which promotes economic expansion.

Policy Implications: When making decisions on monetary and fiscal policy, policymakers keep a careful eye on the unemployment rate. Increased government expenditure or tax breaks are examples of stimulus measures that may be prompted by a high unemployment rate. Interest rate increases and other measures intended to keep the economy from overheating may result from a low unemployment rate.

Labor Force Participation: The unemployment rate is frequently combined with the labor force participation rate because it does not account for people who have given up looking for job. High unemployment and a low participation rate may be signs of economic issues, such as long-term structural unemployment or disillusioned workers quitting the workforce.

32

4

Software Transactional Memory (STM) is a concurrency control solution that enables efficient shared memory management in parallel computing environments. STM's primary goal has been to make it easier to develop multi-threaded systems by enabling a transaction-based approach to memory access control. By ensuring the ACID features in memory transactions, STM can boost the scalability and throughput of systems used for real-time data processing and computations.

This paper examines the possible uses of STM in the context of calculating GDP and unemployment, with a focus on improving computational effectiveness.

GDP and Unemployment rates for a range of years have been calculated simultaneously from "India GDP 1960-2022" dataset accessed from Kaggle. Simultaneous calculation improves effectiveness resulting in faster decision making.

Four columns can be found in "India GDP 1960-2022" dataset: Year, Real GDP, Per Capita in Rupees, and GDP Growth Percentage. It contains information for the year from 1960 to 2021.

2 Literature Survey

21

8

5

20

6

11

19

7

4

33

In their 1990 publication "Software Transactional Memory," Nir Shavit and Dan Touitou presented Software Transactional Memory (STM), a unique software technique that supports adaptable transactional programming of synchronization actions. [1]. Then, in a 1993 paper titled "Transactional memory: architectural support for lock-free data structures," Maurice Herlihy and J. Eliot B. Moss presented transactional memory, a novel multiprocessor architecture designed to make lock-free synchronization as effective (and user-friendly) as traditional methods based on mutual exclusion. [2]. A new type of software transactional memory (STM) that supports dynamic-sized data structures was then proposed by Maurice Herlihy, Victor Luchanogo, Mark Moir, and William N. Scherer in their 2003 paper "Software transactional memory for dynamic-sized data structures," [3]. Next, in a 2010 paper titled "Cloud-TM: harnessing the cloud with distributed transactional memories," Paolo Romano, Luis Rodrigues, Nuno Carvalho, and Jaoa Cachopo talked about the application of the Transactional Memory programming model within the framework of the cloud computing paradigm, which is called Cloud-TM [4]. The issue of how to create speculative replication protocols for transactional systems layered on top of an Optimistic Atomic Broadcast (OAB) service was then examined theoretically in a paper titled "On Speculative Replication of Transactional Systems" published in 2014 by Paolo Romano, Roberto Palmieri, Francesco Quaglia, and Nuno Carvalho [5]. Then, in 2017, M. B. Kordic, I. Basicovic and Popovic in a paper titled "Transaction scheduling for Software Transactional Memory," demonstrated that the Round Robin Algorithm performs worst when there is low contention and dual-core multicore. The Execution

Time-based Load Balancing algorithm outperforms Round Robin Algorithm by an average of 1.12, and the Avoid Conflicts algorithm outperforms Round Robin Algorithm by an average of 1.53—from 1.12 in the worst case to 2.03 in the best case [6]. Then, in their work "Software Transactional Memory with Interactions," published in 2020, Marino Miculan and Marco Peressotti introduced Open Transactional Memory (OTM), a programming abstraction that facilitates secure, data-driven interactions between composable memory transactions[7]. A thorough analysis of Software Transactional Memory (STM) systems was then published by Meenu in 2024 under the title "Software Transactional Memory: A Comprehensive Review of Design, Challenges, Applications, and Future Prospects." This paper focused on the systems' design, evolution, challenges, and applications[8].

We learned about software transactional memory from the aforementioned studies, and have used it in our work.

3 GDP Prediction using Locks and STM

The GDP Growth Rate of a particular year can be calculated by subtracting the real GDP of the year from the real GDP of the previous year. Next, this difference is divided by the previous year's Real GDP.

These days, the synchronization issue is resolved with locks. When many threads of execution are available, a lock, a basic synchronization technique in computer science, limits access to a resource. A mutual exclusion concurrency control strategy is implemented by Lock. The pessimistic execution philosophy is what locks adhere to. They believe that an incorrect result will undoubtedly occur if several threads reach the critical section at the same time. Consequently, they restrict the execution of the critical section to one thread at a time.

Any parallel program that uses locks has to identify and contain the critical section between the `pthread_mutex_lock()` and `pthread_mutex_unlock()` actions in order to maintain synchronization.

Our algorithm is described here.

3.1 Algorithm

Step1: We acquired the "India GDP 1960-2022" dataset from Kaggle. The program moved the CSV file to a multidimensional array.

Step 2: From 1961 to 2021, the Real GDP for the relevant year was entered into a multidimensional array. The array's first column lists the corresponding year, while the array's second column lists each year's Real GDP.

Step 3: Each row's GDP growth is calculated by deducting its real GDP from the real GDP of the row that came before it. The difference is then divided by the Real GDP

6

of the row before it. The outcome, which is the GDP Growth for that particular year, is shown in the third column of that row.

Step 4: We divide the array across the four processors that are available in our scenario.

Step 5: The GDP growth of each processor's part is decided.

3.2 Program using Locks to Calculate GDP Growth

The real GDP of India from 1961 to 2021, together with the corresponding years, are entered into the program using Locks to calculate GDP Growth as a double-dimensional array "a" from the Kaggle dataset "India GDP 1960-2022." Threads are created using the pthread_create() method.

```
for(i=0;i<NUM_THREAD;i++)
{
    arg_to_be_passed[i]=i;
    pthread_create(&tid[i],NULL,thread_find_inf,&arg_to_be_passed[i]);
}
```

The pthread_join() function makes sure that a thread must wait for all other threads to finish, even if it finishes its execution ahead of others.

```
for(i=0;i<NUM_THREAD;i++)
{
    pthread_join(tid[i],NULL);
}
```

The array "a" is split across the available processors in the thread function "thread_find_gdp()."

```
for(i=(((num*n)/(NUM_THREAD)));i<(((num+1)*n)/NUM_THREAD);i++)
```

In order to update the global variable "global_gdp," which keeps track of the total GDP Growths for all years, each processor computes the GDP Growths of its portion.

```
if(i==0)
    continue;
a[i][2]=((a[i]-a[i-1])[1]/a[i-1][1])*100;
pthread_mutex_lock(&mutex1);
global_gdp=global_gdp+a[i][2];
pthread_mutex_unlock(&mutex1);
```

7

The following formula is used to determine a given year's GDP growth.

$$a[i][2] = ((a[i] - a[i-1])[1] / a[i-1][1]) * 100;$$

The following is how the variable "global_gdp" is being updated:

```
global_gdp=global_gdp+a[i][2];
```

The program's critical section, which is surrounded by locks, is when the variable "global_gdp" is updated.

```
pthread_mutex_lock(&mutex1);
global_gdp=global_gdp+a[i][2];
pthread_mutex_unlock(&mutex1);
```

There are no synchronization issues with the software.

The program's output is displayed below.

```
Enter the CSV file name: gdp.csv
Initial data from CSV File:
1961.000000 39.230000 0.000000
1962.000000 42.160000 0.000000
1963.000000 48.410000 0.000000
1964.000000 56.480000 0.000000
1965.000000 59.549999 0.000000
1966.000000 45.069999 0.000000
1967.000000 50.130001 0.000000
1968.000000 53.090000 0.000000
1969.000000 58.450001 0.000000
1970.000000 62.419998 0.000000
1971.000000 67.349998 0.000000
1972.000000 71.459999 0.000000
1973.000000 85.519997 0.000000
1974.000000 99.529999 10.382135
1975.000000 98.470001 -1.055003
1976.000000 102.720001 4.316035
1977.000000 121.489998 18.272972
1978.000000 137.300003 13.013421
1979.000000 152.990005 11.427532
1980.000000 186.330002 21.792271
1981.000000 191.450005 2.742647
1982.000000 208.720001 3.736625
1983.000000 218.259995 4.738538
1984.000000 212.160004 -2.704828
1985.000000 232.509995 9.591813
1986.000000 245.990005 5.807872
1987.000000 279.029999 12.064738
1988.000000 296.509996 6.293229
1989.000000 296.040009 -0.185437
1990.000000 328.980011 8.424538
1991.000000 278.189985 -15.848348
1992.000000 288.209991 3.600976
1993.000000 279.299988 -3.091497
1994.000000 227.279999 -17.178665
1995.000000 368.279999 10.081109
1996.000000 392.099994 6.054068
1997.000000 435.009995 5.846272
1998.000000 421.350001 3.177222
1999.000000 458.020007 8.092884
2000.000000 468.390015 2.085787
2001.000000 465.440002 3.640126
2002.000000 514.940002 6.076961
2003.000000 607.700012 18.013752
2004.000000 709.150024 16.694094
2005.000000 826.300005 15.684971
2006.000000 948.260010 14.612741
2007.000000 1216.739990 29.404631
2008.000000 1198.900024 -1.466210
2009.000000 1341.890015 11.926765
2010.000000 1675.619995 24.870144
2011.000000 1823.050049 8.798537
2012.000000 1827.640015 0.251774
2013.000000 1856.719971 1.591120
2014.000000 2039.130005 9.824316
2015.000000 2103.590088 3.161156
2016.000000 2294.800049 9.089697
2017.000000 2651.469971 15.542526
2018.000000 2782.929932 4.940809
2019.000000 2831.550049 4.758544
2020.000000 2667.689941 -5.786948
2021.000000 3173.399902 18.956848
```

3.3 Program using STM to Calculate GDP Growth

The program that calculates GDP growth using STM has the same structure as the program that uses locks. Lock calls are replaced with only the proper STM calls.

The part of the program that updates the variable "global_gdp" is the critical section, which is why it is encased in the proper STM calls.

```
START (0,RW);
global_gdp=global_gdp+a[i][2];
COMMIT;
```

There are no synchronization issues with the software.

8

The program's output is displayed below.

```
Enter no. of times code is to be executed
1
Initial Data from CSV file
1961.000000 39.230000 0.000000
1962.000000 42.160000 0.000000
1963.000000 45.419998 0.000000
1964.000000 56.480000 0.000000
1965.000000 59.549999 0.000000
1966.000000 45.869999 0.000000
1967.000000 50.130001 0.000000
1968.000000 53.090000 0.000000
1969.000000 58.450001 0.000000
1970.000000 62.419998 0.000000
1971.000000 67.349998 0.000000
1972.000000 71.459999 0.000000
1973.000000 85.519997 0.000000
1974.000000 99.529999 0.000000
1975.000000 95.470001 0.000000
1976.000000 102.720001 0.000000
1977.000000 121.489998 0.000000
1978.000000 127.200003 0.000000
1979.000000 152.990005 0.000000
1980.000000 156.330002 0.000000
1981.000000 153.490005 0.000000
1982.000000 208.720001 0.000000
1983.000000 218.259995 0.000000
1984.000000 212.160004 0.000000
1985.000000 232.589995 0.000000
1986.000000 248.990005 0.000000
1987.000000 279.029999 0.000000
1988.000000 296.589996 0.000000
1989.000000 296.040009 0.000000
1990.000000 328.980011 0.000000
1991.000000 278.109995 0.000000
1992.000000 288.209991 0.000000
1993.000000 279.299998 0.000000
1994.000000 327.279999 0.000000
1995.000000 368.279999 0.000000
1996.000000 392.899994 0.000000
1997.000000 415.069995 0.000000
1998.000000 421.350006 0.000000
1999.000000 458.820007 0.000000
2000.000000 468.390015 0.000000
2001.000000 485.440002 0.000000
2002.000000 514.940002 0.000000
2003.000000 607.780012 0.000000
2004.000000 709.150024 0.000000
2005.000000 820.380005 0.000000
2006.000000 940.260010 0.000000
2007.000000 1216.739999 0.000000
2008.000000 1198.900024 0.000000
2009.000000 1341.890015 0.000000
2010.000000 1075.619995 0.000000
2011.000000 1023.050049 0.000000
2012.000000 1027.640015 0.000000
2013.000000 1856.719971 0.000000
2014.000000 2039.130005 0.000000
2015.000000 2103.590088 0.000000
2016.000000 2294.800049 0.000000
2017.000000 2651.469971 0.000000
2018.000000 2702.929932 0.000000
2019.000000 2831.550049 0.000000
2020.000000 2667.689941 0.000000
2021.000000 3173.399902 0.000000
Final Output
1961.000000 39.230000 0.000000
1962.000000 42.160000 7.468775
1963.000000 48.419998 14.848193
1964.000000 56.480000 16.646817
1965.000000 59.549999 5.435552
1966.000000 45.869999 -22.972294
1967.000000 50.130001 9.287121
1968.000000 53.090000 5.904646
1969.000000 58.450001 10.090665
1970.000000 62.419998 6.792126
1971.000000 67.349998 7.898110
1972.000000 71.459999 6.102451
1973.000000 85.519997 19.675341
1974.000000 99.529999 16.302135
1975.000000 95.470001 -1.005003
1976.000000 102.720001 4.316835
1977.000000 121.489998 18.472972
1978.000000 137.300003 13.013421
1979.000000 152.990005 11.427532
1980.000000 168.230002 21.792271
1981.000000 193.490005 3.842647
1982.000000 208.720001 3.736625
1983.000000 218.259995 8.738538
1984.000000 232.589995 -2.794028
1985.000000 232.589995 9.591813
1986.000000 248.990005 7.087872
1987.000000 279.029999 12.064738
1988.000000 296.589996 6.293229
1989.000000 296.040009 -0.185437
1990.000000 328.980011 8.424538
1991.000000 278.109995 -15.848348
1992.000000 288.209991 6.700976
1993.000000 279.299998 -3.091497
1994.000000 327.279999 17.178655
1995.000000 368.279999 10.083109
1996.000000 392.899994 9.054068
1997.000000 415.069995 5.846272
1998.000000 421.350006 1.317722
1999.000000 458.820007 8.892844
2000.000000 468.390015 2.085787
2001.000000 485.440002 3.640126
2002.000000 514.940002 6.076961
2003.000000 607.780012 18.813752
2004.000000 709.150024 16.694094
2005.000000 820.380005 15.684971
2006.000000 940.260010 14.612741
2007.000000 1216.739999 29.404631
2008.000000 1198.900024 -1.466210
2009.000000 1341.890015 11.926765
2010.000000 1075.619995 24.870144
2011.000000 1023.050049 8.798537
2012.000000 1027.640015 0.251774
2013.000000 1856.719971 1.591120
2014.000000 2039.130005 9.824316
2015.000000 2103.590088 3.161156
2016.000000 2294.800049 9.080697
2017.000000 2651.469971 15.542526
2018.000000 2702.929932 1.940809
2019.000000 2831.550049 4.758544
2020.000000 2667.689941 -5.786940
2021.000000 3173.399902 18.956848
```

31

Software Transactional Memory (STM) is a concurrency control strategy that separates shared-memory operations into transactions that either commit successfully or fail completely.

By using STM, application programmers can avoid the disadvantages of locks, such as deadlock, unnecessary serialization, and priority inversion. Software transactions are memory-based execution units that make it easier for several threads to run simultaneously. Software transactions fix the majority of lock-based system problems. This concept was inspired by database transactions. STM takes an optimistic approach. Unlike lock-based programming, the optimistic STM technique allows a thread to change shared memory without being concerned about what other threads are doing. It records each read and write it makes. After that, the STM system looks at the log to decide whether or not the thread's actions are allowed (called committing) or not (called aborting). This method's benefit is increased concurrency because it removes the need to wait for a resource to become available, enabling several threads to safely and concurrently edit different, usually locked-together regions of a data structure.

29

Any parallel application using STM for synchronization needs to identify and include the critical section between the START and COMMIT transaction calls.

4 Unemployment Calculation using Locks and STM

One way to determine unemployment in a given year is to multiply the difference between GDP growth in the current year and GDP growth in the prior year by -2.

Here is a description of our algorithm.

4.1 Algorithm

Step 1: We acquired the "India GDP 1960-2022" dataset from Kaggle. The program moved the CSV file to a multidimensional array.

Step 2: A multidimensional array containing the GDP growth for the years 1961–2021 and the corresponding year is entered as an input. The array's first column lists the corresponding year, and the second column lists the GDP growth for each year.

Step 3: Each row's unemployment is determined by deducting its GDP growth from the GDP growth of the row that came before it. After that, the difference is multiplied by -2. The outcome, which is the unemployment rate for that particular year, is shown in the third column of that row.

Step 4: The array is divided across the four processors that are available in our scenario.

Step 5: The unemployment rate for each processor's part is decided.

4.2 Program using Locks to Calculate Unemployment

The GDP Growth of India from 1961 to 2021, together with the corresponding years, are entered into the program utilizing Locks to calculate unemployment as a double-dimensional array "a" from the Kaggle Dataset "India GDP 1960-2022." Threads are created using the pthread_create() method.

```
for(i=0;i<NUM_THREAD;i++)
{
    arg_to_be_passed[i]=i;
    pthread_create(&tid[i],NULL,thread_find_inf,&arg_to_be_passed[i]);
}
```

The pthread_join() function makes sure that a thread must wait for all other threads to finish, even if it finishes its execution ahead of others.

```
for(i=0;i<NUM_THREAD;i++)
{
    pthread_join(tid[i],NULL);
}
```

The array "a" is split up across the available processors in the thread function thread_find_unel().

```
for(i=((num*n)/(NUM_THREAD));i<(((num+1)*n)/NUM_THREAD);i++)
```

10

The global variable "global_unel," which holds the total unemployment for all years, is updated by each processor after it has determined the unemployment for its portion.

12

```

        if(i==0)
            continue;
        a[i][2]=((a[i]-a[i-1])[1]/a[i-1][1])*(-2));
        pthread_mutex_lock(&mutex1);
        global_gdp=global_unel+a[i][2];
        pthread_mutex_unlock(&mutex1);
    
```

The following formula is used to determine the unemployment rate for a given year.

18

```

        a[i][2]=((a[i]-a[i-1])[1]/a[i-1][1])*(-2));
    
```

The following is how the variable "global_unel" is being updated.

```

        global_unel=global_unel+a[i][2];
    
```

The program's critical section, which is surrounded by locks, is when the variable "global_unel" is modified.

```

        pthread_mutex_lock(&mutex1);
        global_unel=global_unel+a[i][2];
        pthread_mutex_unlock(&mutex1);
    
```

There are no synchronization issues with the software.

The program's output is displayed below.

11

Updated data after iteration 1:			
1961.000000	3.720000	0.000000	
1962.000000	2.930000	1.580000	
1963.000000	5.990000	-6.119999	
1964.000000	7.450000	-2.920000	
1965.000000	-2.640000	20.180000	
1966.000000	-0.060000	-5.160000	
1967.000000	7.830000	-15.780000	
1968.000000	3.390000	8.879999	
1969.000000	6.540000	-6.300000	
1970.000000	5.160000	2.760000	
1971.000000	1.640000	7.040000	
1972.000000	-0.550000	4.380000	
1973.000000	3.300000	-7.700000	
1974.000000	1.190000	4.220000	
1975.000000	9.150000	-15.919999	
1976.000000	1.660000	14.980000	
1977.000000	7.250000	-11.180000	
1978.000000	5.710000	3.080000	
1979.000000	-5.240000	21.900000	
1980.000000	6.740000	-23.959999	
1981.000000	6.010000	1.459999	
1982.000000	3.480000	5.060000	
1983.000000	7.290000	-7.620000	
1984.000000	3.820000	6.940000	
1985.000000	5.250000	-2.860000	
1986.000000	4.780000	0.940000	
1987.000000	3.970000	1.620000	
1988.000000	9.630000	-11.320000	
1989.000000	5.950000	7.360001	
1990.000000	5.530000	0.839999	
1991.000000	1.060000	8.940001	
1992.000000	5.480000	-8.840000	
1993.000000	4.750000	1.460000	
1994.000000	6.660000	-3.820000	
1995.000000	7.570000	-1.820001	
1996.000000	7.550000	0.040000	
1997.000000	4.050000	7.000000	
1998.000000	6.180000	-4.259999	
1999.000000	8.850000	-5.340001	
2000.000000	3.840000	10.020000	
2001.000000	4.820000	-1.960001	
2002.000000	3.800000	2.040000	
2003.000000	7.860000	-8.120001	
2004.000000	7.920000	-0.120000	
2005.000000	7.920000	-0.000000	
2006.000000	8.060000	-0.280001	
2007.000000	7.660000	0.800001	
2008.000000	3.090000	9.139999	
2009.000000	7.860000	-9.540001	
2010.000000	8.500000	-1.280000	
2011.000000	5.240000	6.520000	
2012.000000	5.460000	-0.440001	
2013.000000	6.390000	-1.860000	
2014.000000	7.410000	-2.040000	
2015.000000	8.000000	-1.180000	
2016.000000	8.260000	-0.520000	
2017.000000	6.800000	2.920000	
2018.000000	6.450000	0.700001	
2019.000000	3.740000	5.420000	
2020.000000	-6.600000	20.680000	
2021.000000	8.950000	-31.099998	

4.3 Program using STM to Calculate Unemployment

The program that calculates unemployment using STM has the same structure as the program that uses locks. Lock calls are replaced with only the proper STM calls.

The part of the program that updates the variable "global_unel" is critical section, which is why it is encased in the proper STM calls.

```
START (0,RW);
global_unel=global_unel+a[i][2];
COMMIT;
```

There are no synchronization issues with the software.

The program's output is displayed below.

12

Enter no. of times code is to be executed	2000.000000 3.840000 0.000000	1980.000000 6.740000 -23.959999
1	2001.000000 4.820000 0.000000	1981.000000 6.010000 1.459999
Initial Data	2002.000000 3.800000 0.000000	1982.000000 3.480000 5.060000
1961.000000 3.720000 0.000000	2003.000000 7.860000 0.000000	1983.000000 7.290000 -7.620000
1962.000000 2.930000 0.000000	2004.000000 7.920000 0.000000	1984.000000 3.820000 6.940000
1963.000000 5.990000 0.000000	2005.000000 7.920000 0.000000	1985.000000 5.250000 -2.860000
1964.000000 7.450000 0.000000	2006.000000 8.060000 0.000000	1986.000000 4.780000 0.940000
1965.000000 -2.640000 0.000000	2007.000000 7.660000 0.000000	1987.000000 3.970000 1.620000
1966.000000 -0.850000 0.000000	2008.000000 3.090000 0.000000	1988.000000 9.630000 -11.320000
1967.000000 7.830000 0.000000	2009.000000 7.860000 0.000000	1989.000000 5.950000 7.360001
1968.000000 3.390000 0.000000	2010.000000 8.500000 0.000000	1990.000000 5.530000 0.839999
1969.000000 6.540000 0.000000	2011.000000 5.240000 0.000000	1991.000000 1.060000 8.940001
1970.000000 5.160000 0.000000	2012.000000 5.460000 0.000000	1992.000000 5.480000 -8.840000
1971.000000 1.640000 0.000000	2013.000000 6.390000 0.000000	1993.000000 4.750000 1.460000
1972.000000 -0.550000 0.000000	2014.000000 7.410000 0.000000	1994.000000 6.660000 -3.820000
1973.000000 3.300000 0.000000	2015.000000 8.000000 0.000000	1995.000000 7.570000 -1.820001
1974.000000 1.190000 0.000000	2016.000000 8.260000 0.000000	1996.000000 7.550000 0.040000
1975.000000 9.150000 0.000000	2017.000000 6.800000 0.000000	1997.000000 4.050000 7.000000
1976.000000 1.660000 0.000000	2018.000000 6.450000 0.000000	1998.000000 6.180000 -4.259999
1977.000000 7.250000 0.000000	2019.000000 3.740000 0.000000	1999.000000 8.850000 -5.340001
1978.000000 5.710000 0.000000	2020.000000 -6.600000 0.000000	2000.000000 3.840000 10.020000
1979.000000 -5.240000 0.000000	2021.000000 8.950000 0.000000	2001.000000 4.820000 -1.960001
1980.000000 6.740000 0.000000	Final Output	2002.000000 3.800000 2.040000
1981.000000 6.010000 0.000000	1961.000000 3.720000 0.000000	2003.000000 7.860000 -8.120001
1982.000000 3.480000 0.000000	1962.000000 2.930000 1.580000	2004.000000 7.920000 -0.120000
1983.000000 7.290000 0.000000	1963.000000 5.990000 -6.119999	2005.000000 7.920000 -0.000000
1984.000000 3.820000 0.000000	1964.000000 7.450000 -2.920000	2006.000000 8.060000 -0.280001
1985.000000 5.250000 0.000000	1965.000000 -2.640000 20.180000	2007.000000 7.660000 0.800001
1986.000000 4.780000 0.000000	1966.000000 -0.850000 -5.160000	2008.000000 3.090000 9.139999
1987.000000 7.830000 0.000000	1967.000000 7.830000 -15.780000	2009.000000 7.860000 -9.840001
1988.000000 9.630000 0.000000	1968.000000 3.390000 8.879999	2010.000000 8.500000 -1.280000
1989.000000 5.950000 0.000000	1969.000000 6.540000 -6.300000	2011.000000 5.240000 6.520000
1990.000000 5.530000 0.000000	1970.000000 5.160000 2.760000	2012.000000 5.460000 -0.440001
1991.000000 1.060000 0.000000	1971.000000 1.640000 7.040000	2013.000000 6.390000 -1.860000
1992.000000 5.480000 0.000000	1972.000000 -0.550000 4.380000	2014.000000 7.410000 -2.040000
1993.000000 4.750000 0.000000	1973.000000 3.300000 -7.700000	2015.000000 8.000000 -1.180000
1994.000000 6.660000 0.000000	1974.000000 1.190000 4.220000	2016.000000 8.260000 -0.520000
1995.000000 7.570000 0.000000	1975.000000 9.150000 -15.919999	2017.000000 6.800000 2.920000
1996.000000 7.550000 0.000000	1976.000000 1.660000 14.980000	2018.000000 6.450000 0.700001
1997.000000 4.050000 0.000000	1977.000000 7.250000 -11.180000	2019.000000 3.740000 5.420000
1998.000000 6.180000 0.000000	1978.000000 5.710000 3.080000	2020.000000 -6.600000 20.680000
1999.000000 8.850000 0.000000	1979.000000 -5.240000 21.900000	2021.000000 8.950000 -31.099998

STM calls are used instead of lock calls, and the program's structure remains the same as when locks are used. There are no problems with synchronization.

5 Performance Comparisons of Locks and STM

In this work, the Kaggle Dataset "India_GDP_1960-2022" was used to compute GDP growth and the unemployment rate using Locks and STM. For a variety of years, the GDP and unemployment rates have been computed concurrently. Effectiveness is increased through simultaneous computation, which speeds up decision-making. STM is not affected by any lock-related issues. STM performs better because it employs optimistic execution as opposed to locks' pessimistic execution. This allows for the simultaneous operation of several threads. By substituting the proper STM calls for lock calls, lock-based parallel programs can likewise be converted to STM-based parallel programs. Therefore, STM's compatibility with older codes is one of its main advantages. Another benefit of STM is that it can be used without requiring us to select the critical section for our subsequent study. As a result, it is reasonable to predict that STM will soon be extensively utilized in parallel programming.

6 How STM improves GDP and Unemployment Prediction

Applications of STMs to enhance computational model performance and accuracy may indirectly improve forecasts of GDP and unemployment. In the perspective of forecasting GDP and unemployment, STM may be useful in the following ways:

1. Economic models can operate more quickly and efficiently thanks to STM's enhanced computational efficiency. For instance, STM might aid in expediting the pro-

cessing of several variables that affect GDP and unemployment in extensive macroeconomic simulations. This makes it possible to include real-time data and produce forecasts that are more accurate.

2. A lot of forecasting models must simultaneously examine data from several sources and variables. STM can streamline the management of these simultaneous processes, improving the efficiency and dependability of the modeling process.

3. STM can enable more intricate and nuanced forecasts of future economic outcomes by executing several scenarios concurrently. Both short- and long-term projections for GDP and unemployment may become more accurate as a result.

4. STM makes it easier to update GDP and unemployment projections as new economic data becomes available. For forecasting and policy-making to be prompt and responsive, real-time data integration is essential. This enables decision-makers to respond to developments more confidently and quickly.

5. STM's safe concurrency management makes GDP and unemployment projections less vulnerable to errors that may result from computation faults or race situations. Better decision-making may result from policymakers having greater faith in the findings.

6. STM makes it possible for more reliable economic models that can replicate a variety of variables and situations by effectively employing computational resources. As a result, computations for GDP and unemployment may become more comprehensive and precise.

7. STM can assist in coordinating distributed models that estimate GDP and unemployment in large-scale economic forecasting systems, guaranteeing that the system functions cohesively and effectively even when several nodes access and process data at the same time.

7 Conclusion

In comparing the calculation of GDP and unemployment using traditional **locks** versus **Software Transactional Memory (STM)**, it is evident that each approach has its strengths and trade-offs in the context of concurrent computing. The **lock-based** approach, while simpler to implement, often faces challenges with scalability and performance in multi-threaded environments. Locks can introduce significant over-

14

head due to contention, leading to deadlocks, especially when handling large, complex datasets typical in economic forecasting.

On the other hand, STM provides a more sophisticated and efficient solution for concurrent programming. STM allows for safe and automatic handling of data access and updates across multiple threads without the need for explicit locks. This results in improved scalability, reduced contention, and faster execution, especially in large-scale simulations of GDP and unemployment computations. By leveraging STM's ability to handle parallel transactions atomically and efficiently, economic models can process complex data faster, with greater accuracy and fewer errors.

Ultimately, STM offers a more scalable and robust solution for complex economic modeling tasks like GDP and unemployment computing. While lock-based approaches may suffice for simpler applications, STM provides a significant advantage when it comes to handling real-time data integration, running large simulations, and ensuring data consistency across multiple processes. As the demand for more advanced and responsive economic computations grows, STM is likely to play an increasingly important role in optimizing computation models and enhancing the reliability of computations. GDP and Unemployment rates for a range of years have been calculated simultaneously in this paper. Simultaneous calculation improves effectiveness resulting in faster decision making. STM also has the advantage of being applicable without requiring us to choose the critical section that will be the subject of our future research.

References

- [1] Nir Shavit, Dan Touitou, "Software Transactional Memory", 1990
- [2] Herlihy, M., & Moss, J. E. B. (1993). Transactional memory. *ACM SIGARCH Computer Architecture News*, 21(2), 289–300. <https://doi.org/10.1145/173682.165164>
- [3] Maurice Herlihy, Victor Luchango, Mark Moir, William N. Scherer, "Software transactional memory for dynamic-sized data structures", *PODC '03: Proceedings of the twenty-second annual symposium on Principles of distributed computing*, pp 92 – 101, 2003
- [4] Marino Miculan and Marco Peressotti, "Software Transactional Memory with Interactions", *CEUR Workshop Proceedings*, pp 67-80, 2005

 4

 30

 13

 23

 3

 3

- [5] Paolo Romano, Luis Rodrigues, Nuno Carvalho, Jaoa Cachopo, "Cloud-TM: harnessing the cloud with distributed transactional memories", 2010
- [6] Paolo Romano, Roberto Palmieri, Francesco Quaglia, Nuno Carvalho, "On speculative replication of transactional systems", *Journal of Computer and System Sciences* Volume 80, Issue 1, pp 257-276, February 2014
- [7] M. Popovic, B. Kordic and I. Basicovic, "Transaction scheduling for Software Transactional Memory" (2017, April 1). *IEEE Conference Publication | IEEE Xplore*. <https://ieeexplore.ieee.org/document/7951909/>
- [8] Miculan, M., & Peressotti, M. (2020, July 17). Software Transactional Memory with Interactions. *arXiv.org*. <https://arxiv.org/abs/2007.10809>,
- [9] Meenu. (2024, June 12). Software Transactional Memory: A comprehensive review of design, challenges, applications, and future prospects. <https://ijisae.org/index.php/IJISAE/article/view/7323>